

Compte-rendu de TP

Interface Web & outils en ligne de commande — Proxmox VE 9

Domaine : Linux / Systèmes · Niveau : Intermédiaire · Durée prévue : 1 h Date de réalisation : 22/09/2026 · Durée réelle : 14 h 44 – 15 h 33 (≈ 50 min)

1. Contexte et objectifs

Ce TP porte sur la gestion de machines virtuelles sous **Proxmox VE 9**, par ses deux interfaces : l'interface web et les outils en ligne de commande. Le scénario professionnel est celui d'un technicien qui doit provisionner rapidement un environnement de test, puis en ajuster les ressources — d'abord manuellement, ensuite de façon automatisable.

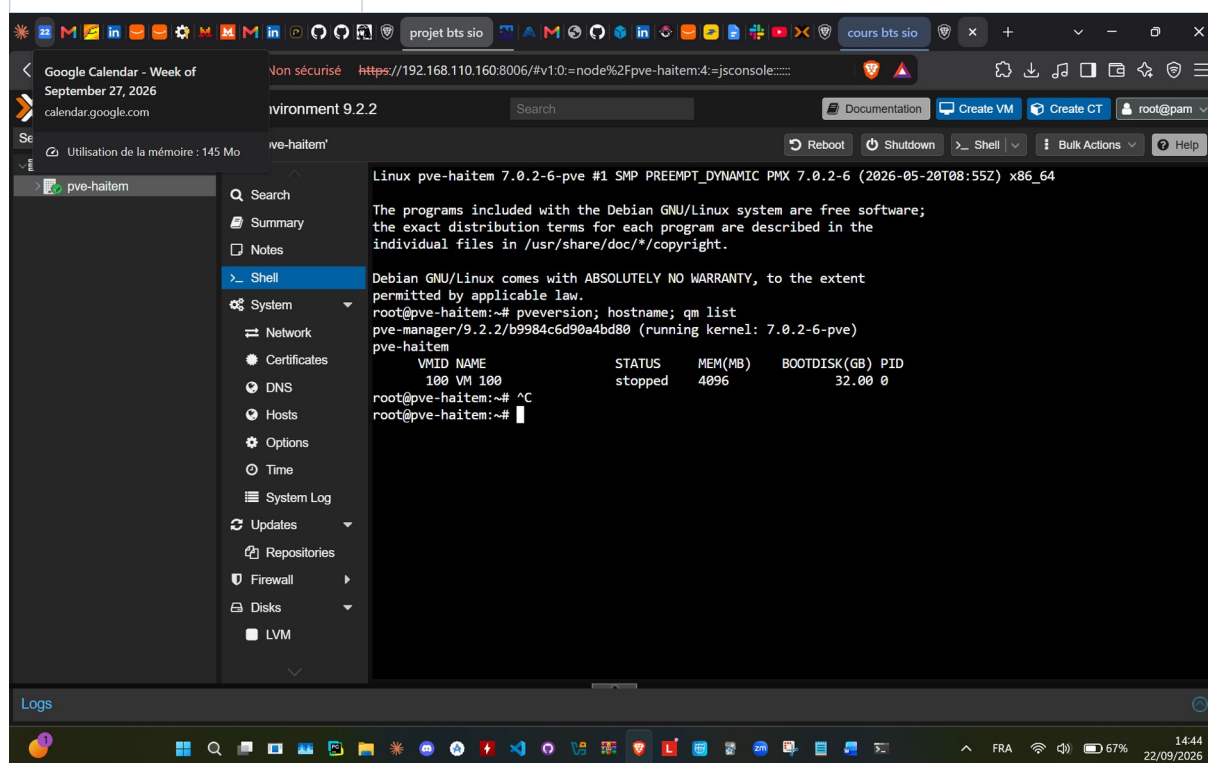
Trois exercices :

1. Créer, démarrer, arrêter et supprimer une VM depuis l'interface web
2. Piloter cette VM en ligne de commande
3. Modifier ses ressources à chaud et gérer ses snapshots

2. Environnement de travail

Élément	Valeur relevée
Hyperviseur	Proxmox Virtual Environment 9.2.2
Noyau	7.0.2-6-pve (Debian GNU/Linux)
Gestionnaire	pve-manager/9.2.2/b9984c6d90a4bd80
Nom du nœud	pve-haitem
Adresse de l'interface	https://192.168.110.160:8006

Élément	Valeur relevée
Compte utilisé	root@pam
Stockages	local (ISO, sauvegardes) et local-lvm (disques de VM)
VM préexistante	VMID 100 — non modifiée
VM créée pour le TP	VMID 101, nom TestVM1



Capture 01 — `pveversion`, `hostname` et `qm list` : état du serveur avant le TP.

Remarque préalable sur l'accès

L'accès SSH au serveur (`ssh root@192.168.110.160`) n'a pas pu être établi. Le TP a donc été réalisé via le **Shell intégré à l'interface web** (`pve-haitem` → **Shell**), qui fournit un terminal root strictement équivalent. Cette solution s'est révélée plus pratique : l'interface graphique et le terminal restent côte à côte, ce qui facilite la comparaison des deux méthodes.

3. Exercice 1 — Création et suppression d'une VM via l'interface web

3.1 Objectif

Maîtriser le cycle de vie d'une machine virtuelle depuis l'interface graphique : création, démarrage, arrêt, suppression.

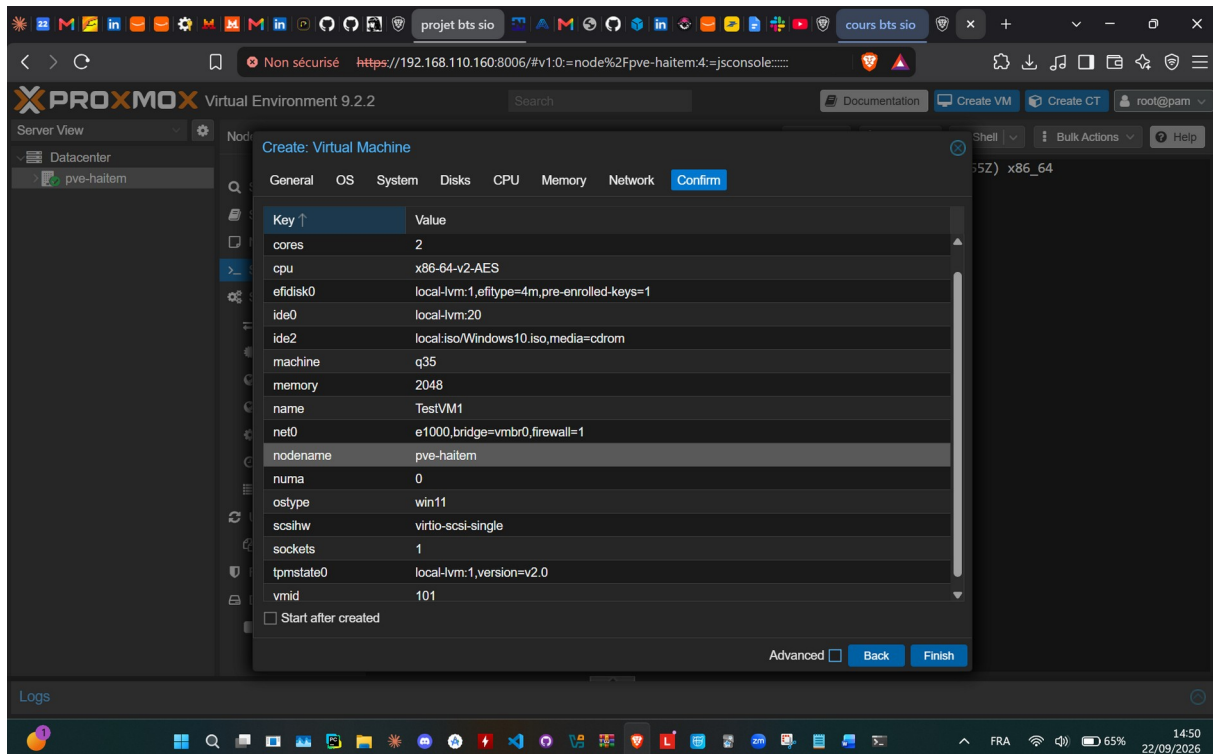
3.2 Choix techniques

L'énoncé demande de « sélectionner un modèle (ex : Debian 11) ». Proxmox ne fonctionne pas ainsi : la création d'une VM exige une **image ISO**. Les seules images disponibles sur le serveur étant Windows 10 et Windows 11, **Windows 10** a été retenu — Windows 11 impose TPM 2.0, Secure Boot et UEFI, contraintes qui auraient alourdi le TP sans rien apporter.

L'installation du système d'exploitation n'a volontairement **pas** été menée à son terme : l'objectif est de valider que la VM démarre, pas de disposer d'un poste Windows.

3.3 Paramètres de création

Onglet	Paramètre	Valeur
General	Node / VM ID / Name	pve-haitem / 101 / TestVM1
OS	ISO image	local:iso/Windows10.iso
OS	Type	Microsoft Windows, version 11/2022/2025
System	BIOS / Machine	OVMF (UEFI) / q35
Disks	Bus / Storage / Taille	IDE 0 / local-lvm / 20 GiB
CPU	Sockets × Cores	1 × 2 = 2 vCPU
Memory	Memory	2048 MiB
Network	Bridge / Modèle	vmbr0 / Intel E1000

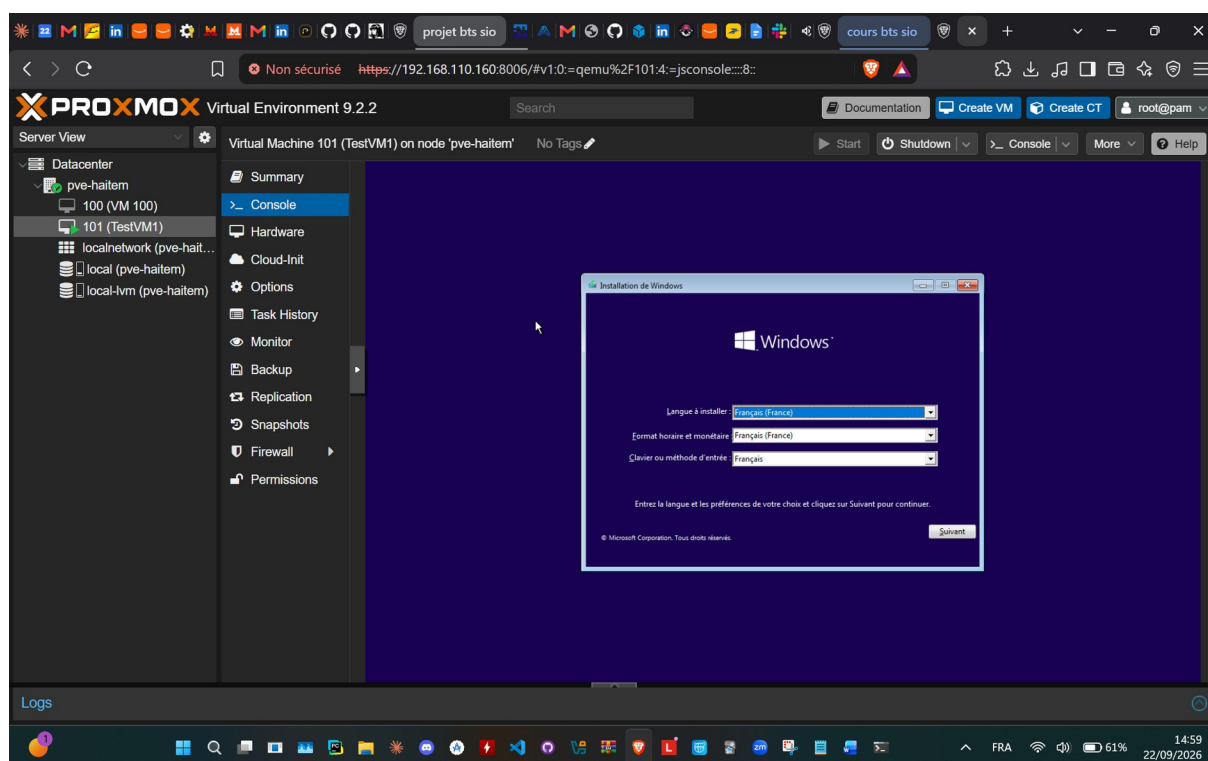


Capture 02 — onglet *Confirm* de l'assistant, avant validation.

Observation importante : le choix du profil Windows 11 a déclenché la création automatique de deux disques supplémentaires de 4 Mo — `efidisk0` (variables du firmware UEFI) et `tpmstate0` (état du module TPM 2.0 virtuel). Proxmox adapte donc le matériel virtuel au système déclaré : le champ `OS Type` n'est pas une simple étiquette, il conditionne la configuration matérielle.

3.4 Démarrage

Après `Start`, la console noVNC affiche l'écran d'installation de Windows 10 (sélection de la langue). La VM démarre donc correctement.



Capture 03 — console noVNC et pastille verte sur 101 (TestVM1) : VM en cours d'exécution.

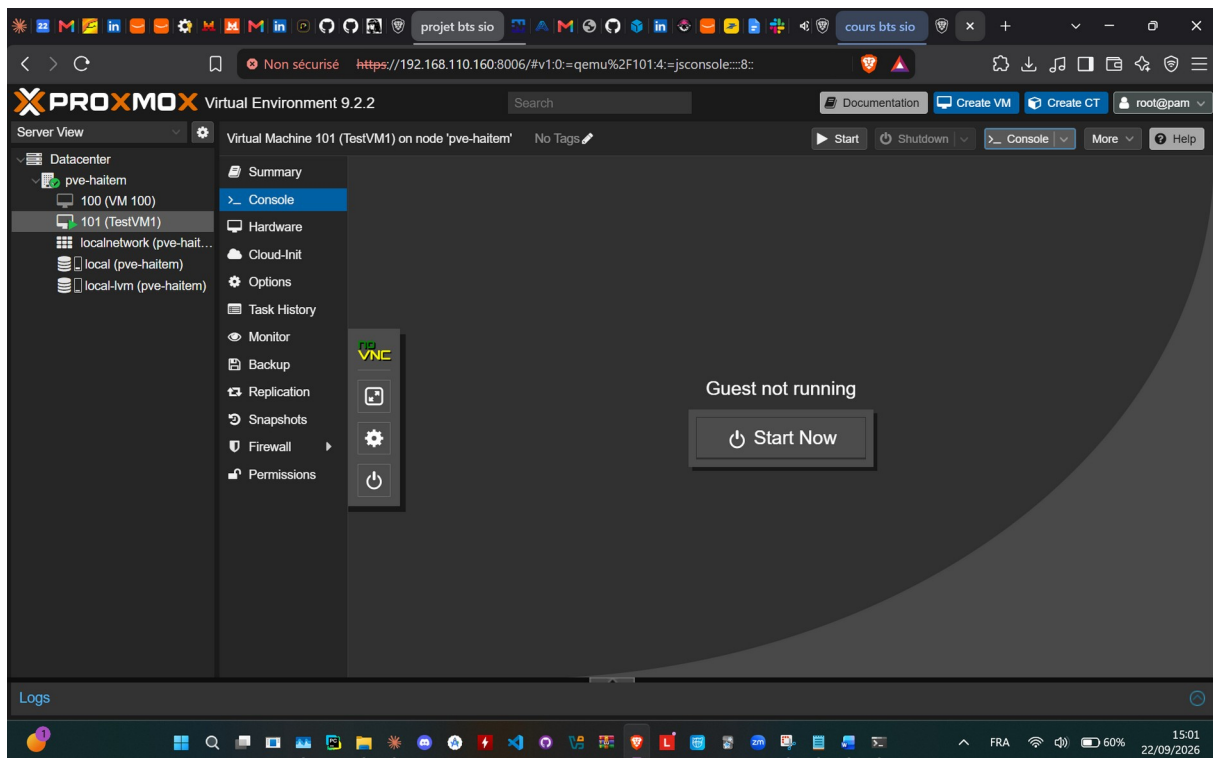
L'énoncé attendait que « la VM démarre et affiche un bureau ». C'est impossible sur une machine neuve : aucun système d'exploitation n'est installé sur le disque. Le critère réellement vérifiable est l'amorçage sur le support d'installation, ce qui est atteint ici.

3.5 Arrêt

Deux options ont été testées depuis l'interface :

Option	Mécanisme	Résultat
Shutdown	Signal ACPI envoyé au système invité	✗ sans effet
Stop	Arrêt du processus QEMU côté hyperviseur	✓ arrêt immédiat

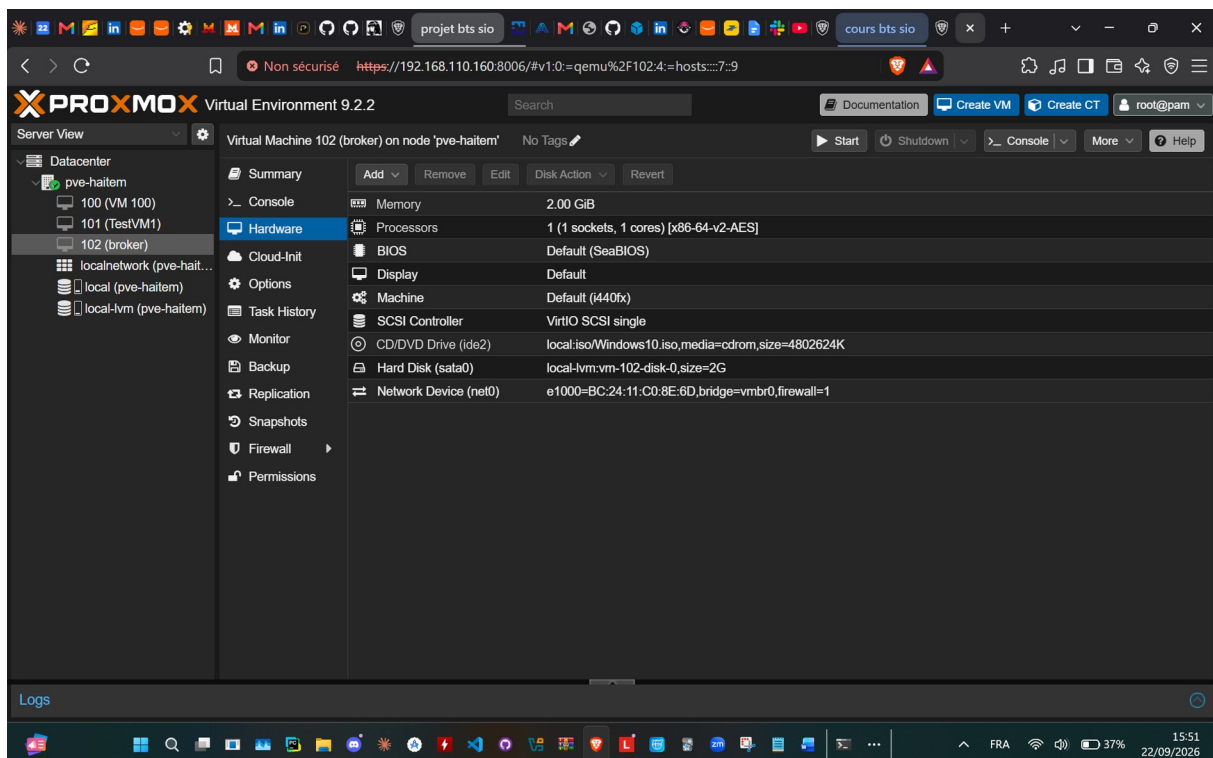
Analyse : l'arrêt ACPI suppose qu'un système d'exploitation écoute le signal et déclenche sa propre procédure d'extinction. L'installateur Windows chargé en mémoire ne remplit pas ce rôle. **Stop** agit à un autre niveau — il tue le processus QEMU sur l'hôte — et fonctionne donc toujours, au prix d'une coupure brutale (équivalent d'une coupure de courant, avec le risque de corruption que cela implique sur un système en production).



Capture 04 — *Guest not running*, bouton *Start* réactivé : la VM est arrêtée.

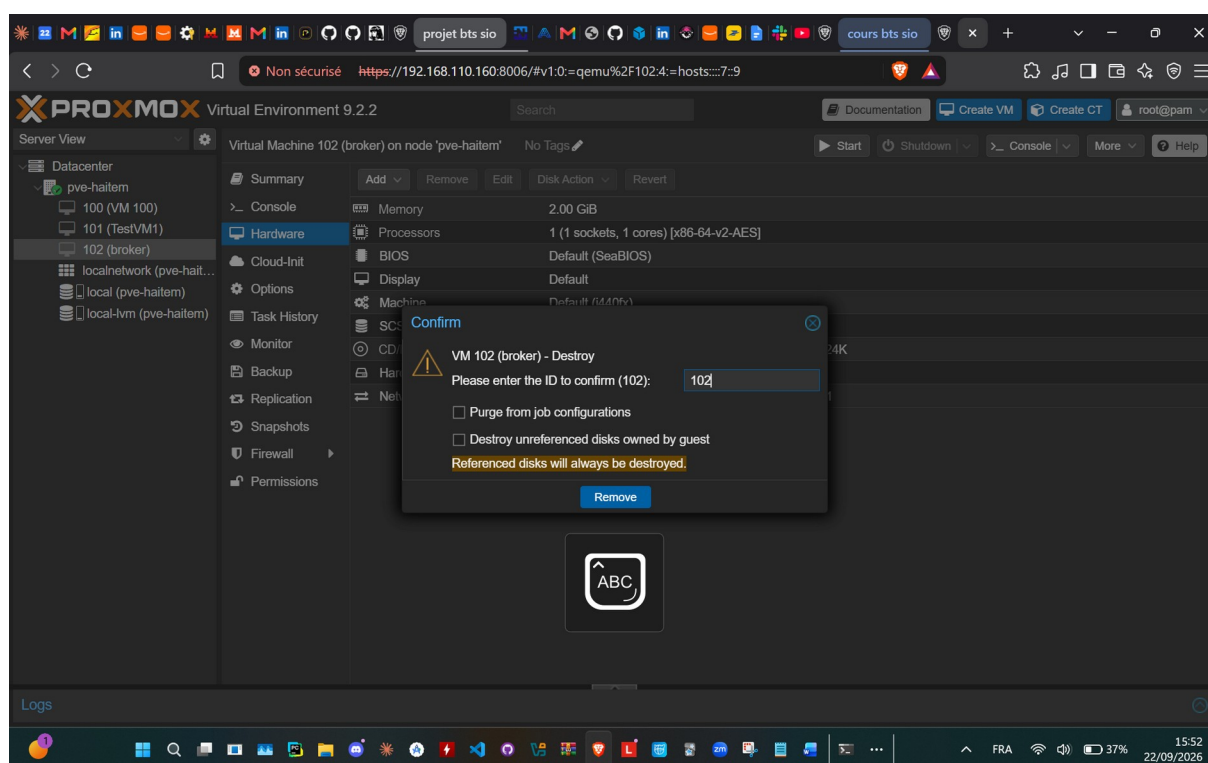
3.6 Suppression

La VM 101 devant servir de source de sauvegarde au TP03, la suppression a été démontrée sur une machine dédiée à cet usage : une VM jetable 102 / *broker* (SeaBIOS, i440fx, 2 Gio de RAM, 1 vCPU, disque *sata0* de 2 Go) créée puis détruite.



Capture 16a — configuration de la VM 102 (*broker*) avant suppression.

Procédure : 102 (broker) → More → Remove.



Capture 16b — dialogue de confirmation, VMID saisi et options cochées.

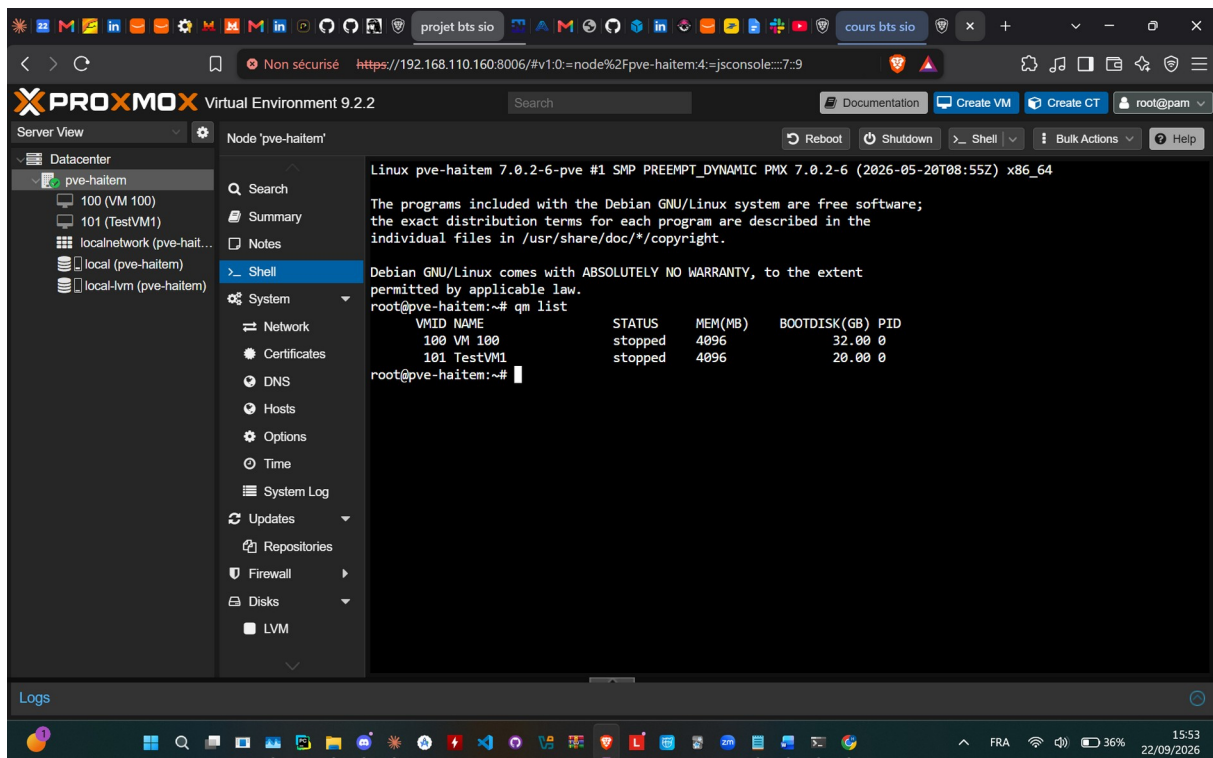
Analyse des garde-fous. Proxmox impose une double confirmation avant destruction :

1. **Retaper le VMID** dans un champ de saisie — un simple clic accidentel ne suffit pas.
2. Un avertissement en surbrillance : « **Referenced disks will always be destroyed** ».

Cet avertissement est le point à retenir : le disque référencé dans la configuration (**sata0** ici) est détruit **dans tous les cas**, indépendamment des cases à cocher. Celles-ci ne concernent que ce qui se trouve *en dehors* de la configuration :

Option	Portée
Purge from job configurations	Retire la VM des tâches de sauvegarde et de réplication, pour éviter des jobs qui échoueraient ensuite sur une VM inexistante
Destroy unreferenced disks owned by guest	Détruit les disques <i>orphelins</i> appartenant à la VM — un disque détaché de la configuration mais jamais effacé du stockage

Les deux ont été cochées : c'est la bonne pratique, sous peine de laisser des résidus sur le stockage et des tâches planifiées pointant dans le vide.



Capture 17 — après suppression, il ne reste que **100** et **101**.

Cette dernière capture confirme au passage que la modification de l'exercice 3 a bien persisté : **101 TestVM1** apparaît désormais avec **MEM(MB) 4096** au lieu de 2048.

3.7 Critères de validation

- ☒ La VM **TestVM1** est créée avec les paramètres définis
- ☒ La VM démarre (amorçage sur l'ISO vérifié en console)
- ☒ La VM est arrêtée via l'interface web
- ☒ La VM est supprimée via l'interface web (démontré sur la VM **102**, **101** étant conservée pour le TP03)

4. Exercice 2 — Gestion des VM en ligne de commande

4.1 Objectif

Réaliser les mêmes opérations qu'à l'exercice 1, mais en ligne de commande — situation d'un administrateur qui doit scripter ses interventions.

4.2 Correction préalable de l'énoncé

L'énoncé s'appuie sur une commande **pvecli** / **pvevm** qui n'existe pas dans Proxmox VE. La table de correspondance ci-dessous a servi de base au TP :

Énoncé (inexistant)	Commande réelle	Rôle
<code>pvevm list</code>	<code>qm list</code>	Lister les VM
<code>pvevm start TestVM1</code>	<code>qm start 101</code>	Démarrer
<code>pvevm stop TestVM1</code>	<code>qm stop 101 / qm shutdown 101</code>	Arrêter (forcé / propre)
<code>pvevm status TestVM1</code>	<code>qm status 101 [--verbose]</code>	Consulter le statut
<code>pvevm list --detail</code>	<code>qm config 101 + qm status 101 --verbose</code>	Obtenir le détail

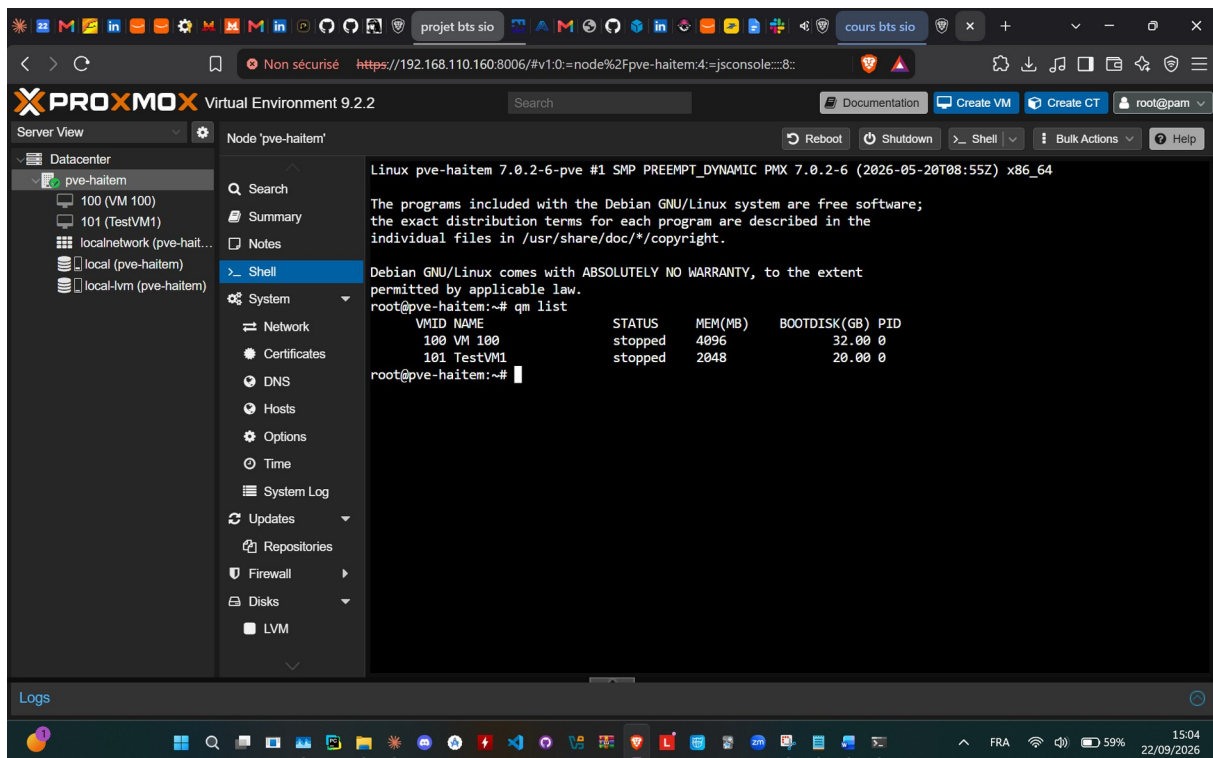
Deux différences de fond, et non de simple orthographe :

1. L'outil s'appelle **qm** (QEMU Manager). Ses équivalents sont **pct** pour les conteneurs LXC, **pvesm** pour le stockage, **pvecm** pour le cluster et **pvesh** pour l'API.
2. **Une VM se désigne par son VMID numérique, jamais par son nom.** `qm start TestVM1` échoue ; `qm start 101` fonctionne. Le nom n'est qu'une étiquette d'affichage.

4.3 Lister les VM

```
qm list
```

VMID	NAME	STATUS	MEM(MB)	BOOTDISK(GB)	PID
100	VM 100	stopped	4096	32.00	0
101	TestVM1	stopped	2048	20.00	0

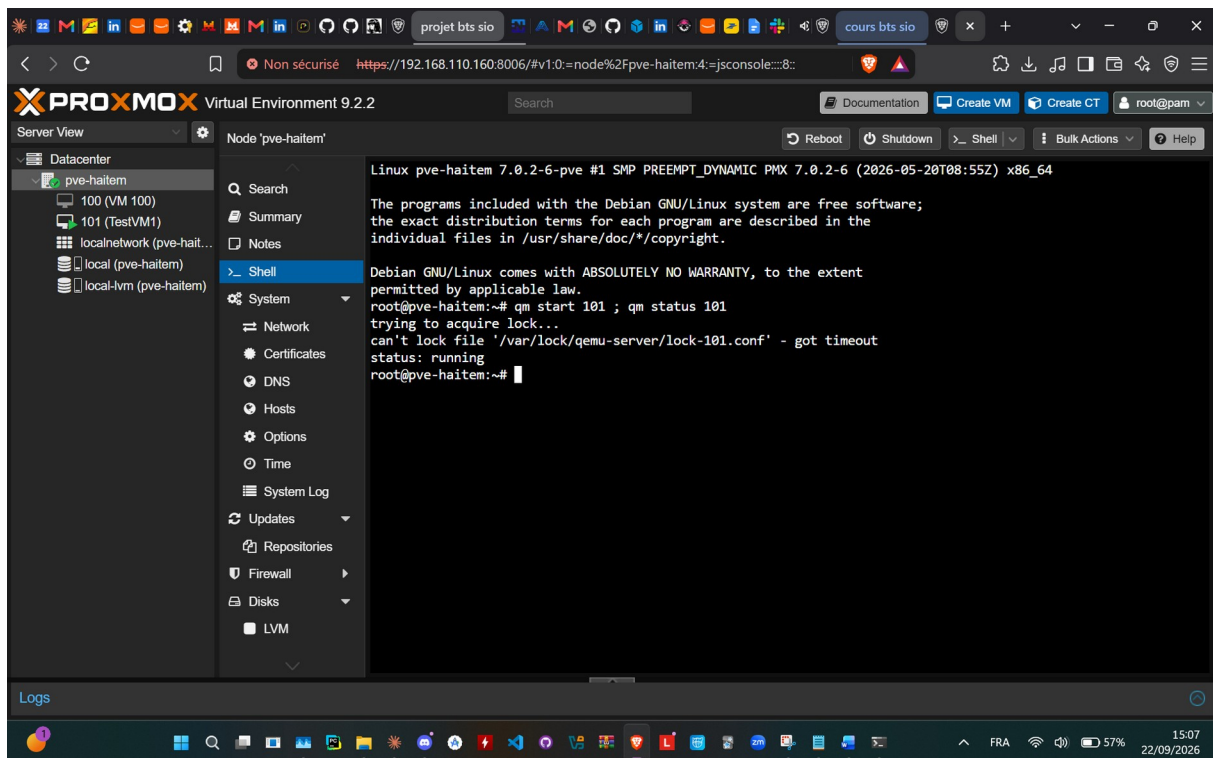


Capture 05 — `qm list`.

4.4 Incident rencontré : verrou de configuration

Le premier `qm start 101` a échoué :

```
trying to acquire lock...
can't lock file '/var/lock/qemu-server/lock-101.conf' - got timeout
status: running
```

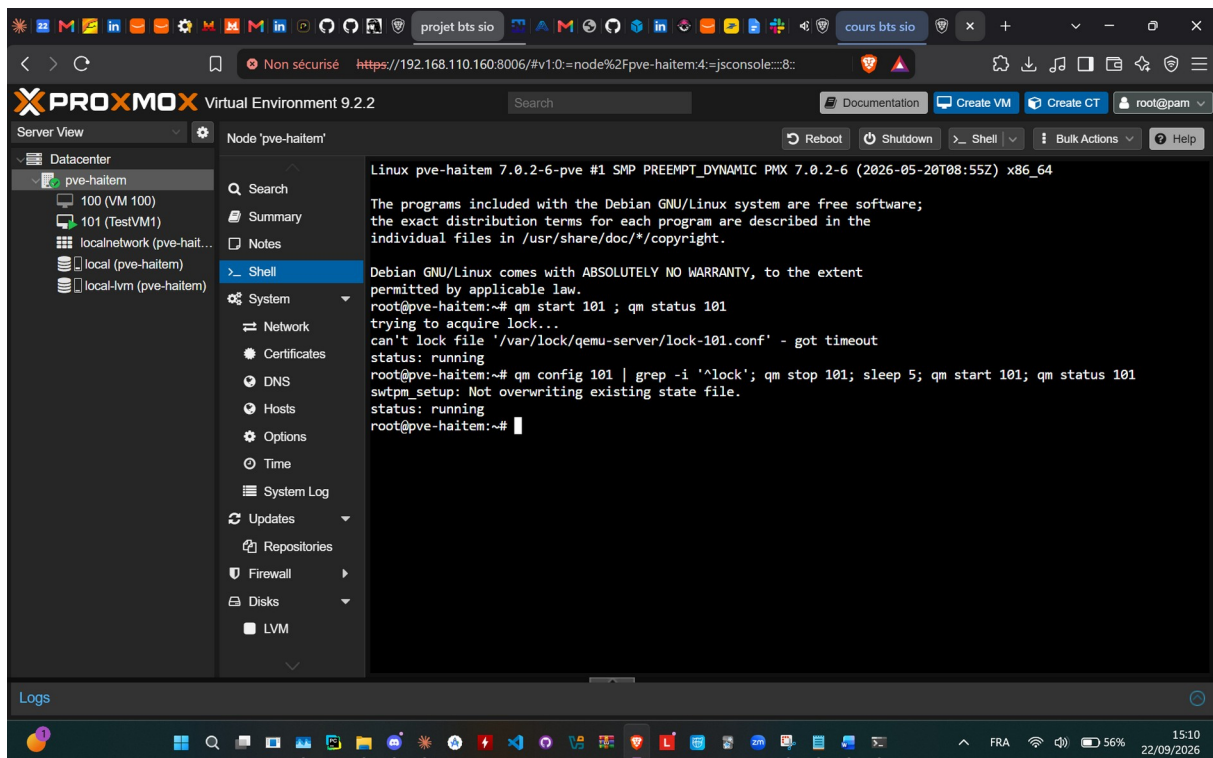


Capture 06 — échec d'acquisition du verrou.

Analyse : Proxmox protège chaque VM par un verrou (**lock**) pour garantir qu'une seule opération modifie sa configuration à la fois. Une autre tâche — un démarrage lancé depuis l'interface web — détenait ce verrou. La commande a attendu 10 secondes puis abandonné. La VM tournait bien, mais **pas du fait de la commande**, ce qui ne validait pas le critère.

Résolution : vérification qu'aucun verrou persistant n'était inscrit dans la configuration, puis reprise complète du cycle en ligne de commande :

```
qm config 101 | grep -i '^lock'    # aucune sortie : pas de verrou bloqué
qm stop 101                        # arrêt en CLI
sleep 5                            # libération du verrou
qm start 101                       # démarrage en CLI
qm status 101                      # vérification
swtvm_setup: Not overwriting existing state file.
status: running
```



Capture 07 — arrêt puis démarrage réussis en ligne de commande.

Deux points de lecture :

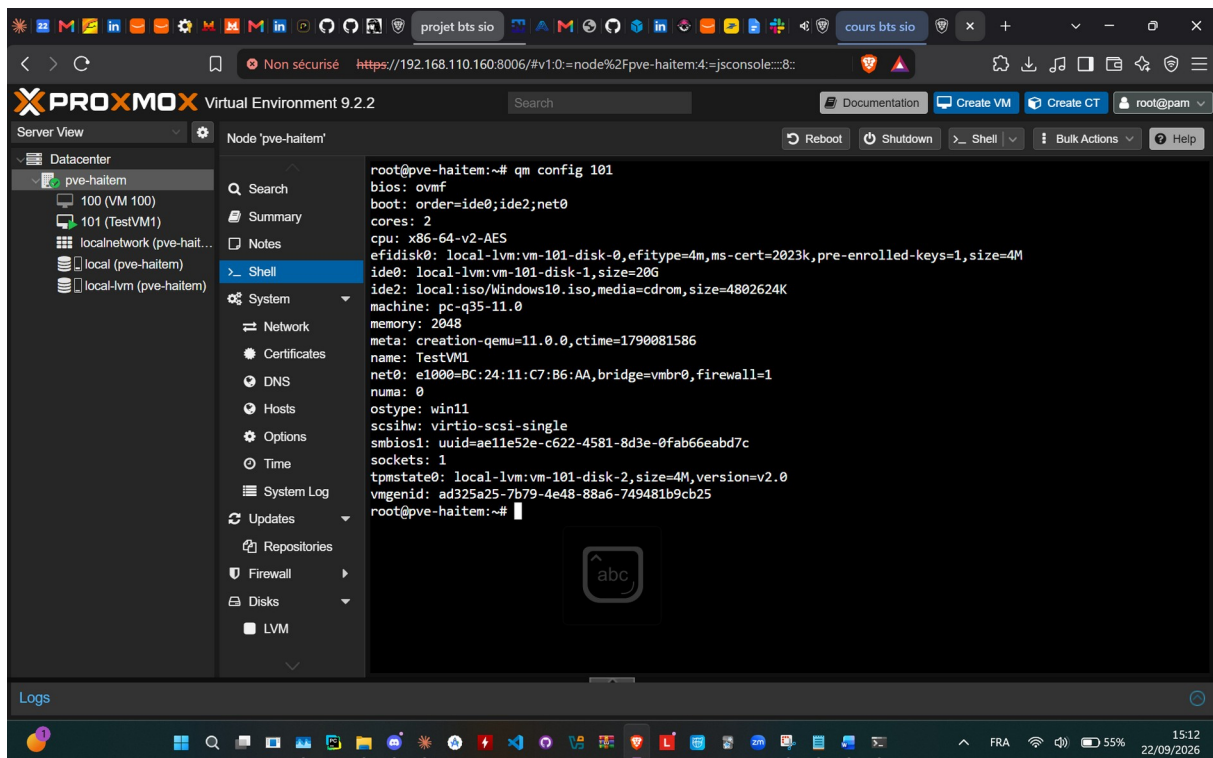
- `qm stop 101` n'affiche **rien** : en environnement Unix, l'absence de sortie signale le succès.
- `swtpm_setup: Not overwriting existing state file.` est un message **informatif** du TPM logiciel, qui réutilise son état existant plutôt que d'en générer un neuf. Ce n'est pas une erreur.

En cas de verrou réellement bloqué (interruption d'une sauvegarde, par exemple), la commande de déblocage est `qm unlock <VMID>`.

4.5 Obtenir le détail d'une VM

L'option `--detail` de l'énoncé n'existe pas. Le détail se répartit en fait sur **deux commandes qui ne répondent pas à la même question** :

Commande	Nature	Source
<code>qm config 101</code>	ce que la VM est (statique)	fichier <code>/etc/pve/qemu-server/101.conf</code>
<code>qm status 101 --verbose</code>	ce que la VM fait (dynamique)	processus QEMU en cours

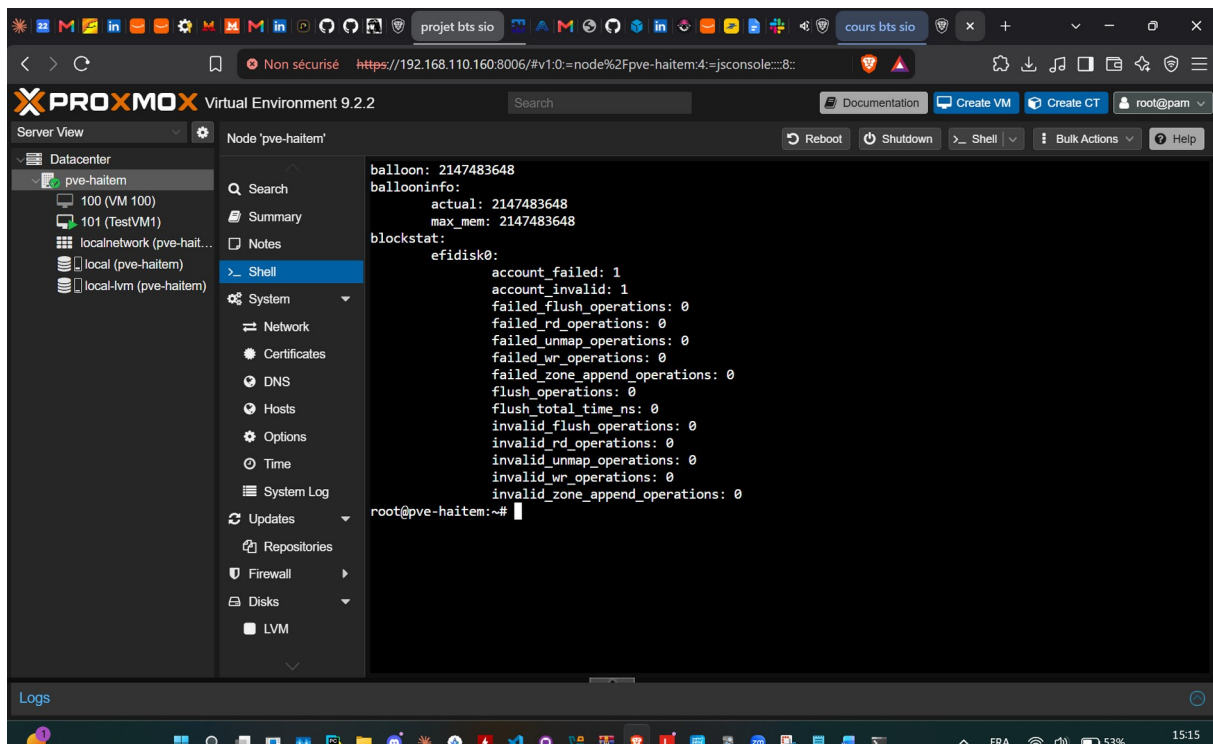


Capture 08 — `qm config 101`.

Relevés notables dans la configuration :

- `boot: order=ide0;ide2;net0` → ordre d'amorçage disque → CD → réseau. Le disque étant vide, le firmware est passé au CD : c'est ce qui a lancé l'installateur Windows.
- `efidisk0` et `tpmstate0` en `size=4M` → les deux disques créés par le profil Windows 11.
- `ide0: local-lvm:vm-101-disk-1,size=20G` → le disque système de 20 Gio.

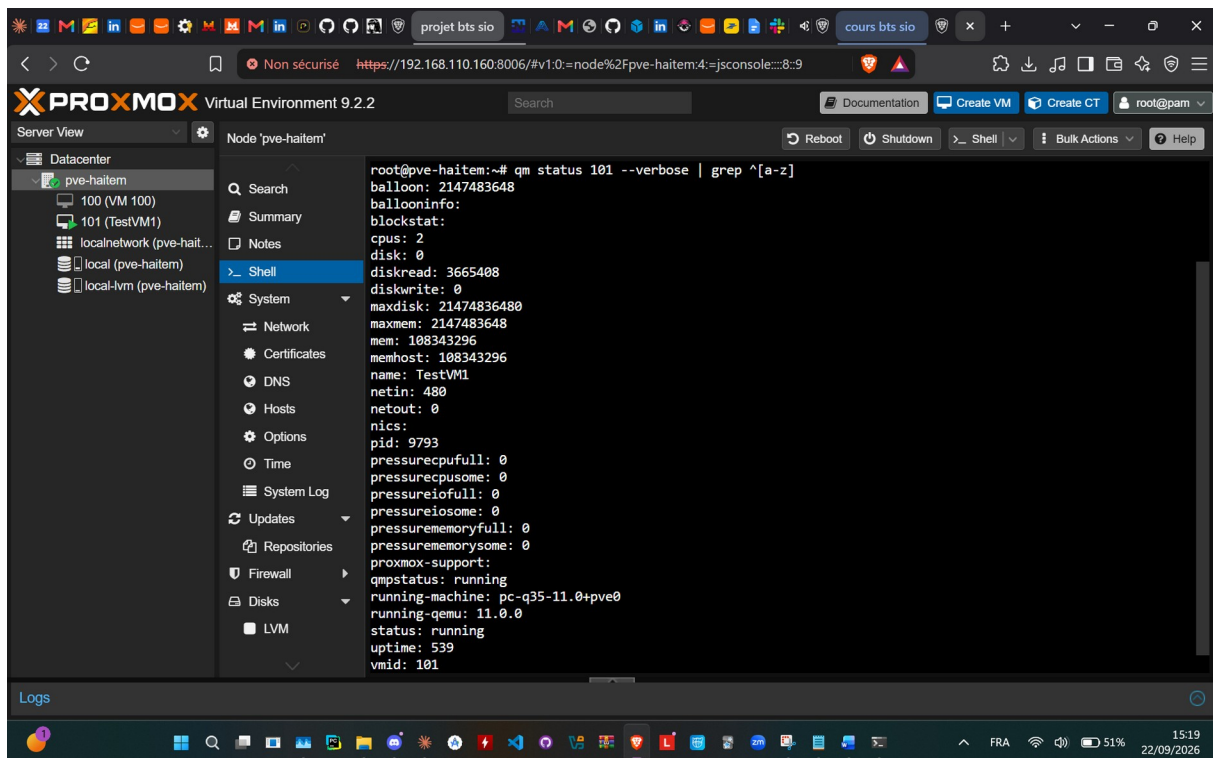
Concernant l'état d'exécution, un premier essai avec `| head -20` s'est révélé inexploitable : la sortie de `--verbose` compte environ 200 lignes **triées alphabétiquement**, si bien que les 20 premières ne contenaient que les statistiques de blocs (`blockstat`).



Capture 09a — `--verbose | head -20` : uniquement des sous-sections, sans intérêt ici.

Un filtrage sur les champs de premier niveau donne le résultat attendu :

```
qm status 101 --verbose | grep ^[a-z]
```



Capture 09b — champs principaux de l'état d'exécution.

Champ	Valeur	Interprétation
<code>status / qmpstatus</code>	<code>running</code>	VM active
<code>cpus</code>	<code>2</code>	1 socket × 2 cœurs
<code>maxmem</code>	<code>2147483648</code>	2 Gio alloués
<code>mem</code>	<code>108343296</code>	≈ 103 Mio réellement consommés
<code>maxdisk</code>	<code>21474836480</code>	20 Gio de disque
<code>pid</code>	<code>9793</code>	PID du processus QEMU sur l'hôte
<code>uptime</code>	<code>539</code>	539 s d'exécution
<code>running-qemu</code>	<code>11.0.0</code>	version de QEMU

Analyse de l'écart `mem` / `maxmem` : 2 Gio sont déclarés, 103 Mio sont effectivement occupés sur l'hôte. C'est le principe de **l'allocation paresseuse** : l'hyperviseur ne réserve physiquement la mémoire qu'au fur et à mesure que la VM y accède. C'est ce qui rend possible le surprovisionnement — allouer au total plus de RAM aux VM que la machine n'en possède. Le périphérique de *ballooning*, complémentaire, permettrait à l'hôte de **reprendre** de la mémoire à une VM, mais il exige un pilote installé dans le système invité, absent ici.

4.6 Critères de validation

- ☒ `qm list` affiche la liste des VM
 - ☒ `qm start 101` démarre la VM
 - ☒ `qm stop 101` arrête la VM
 - ☒ `qm status 101` affiche le statut de la VM
-

5. Exercice 3 — Ressources et snapshots

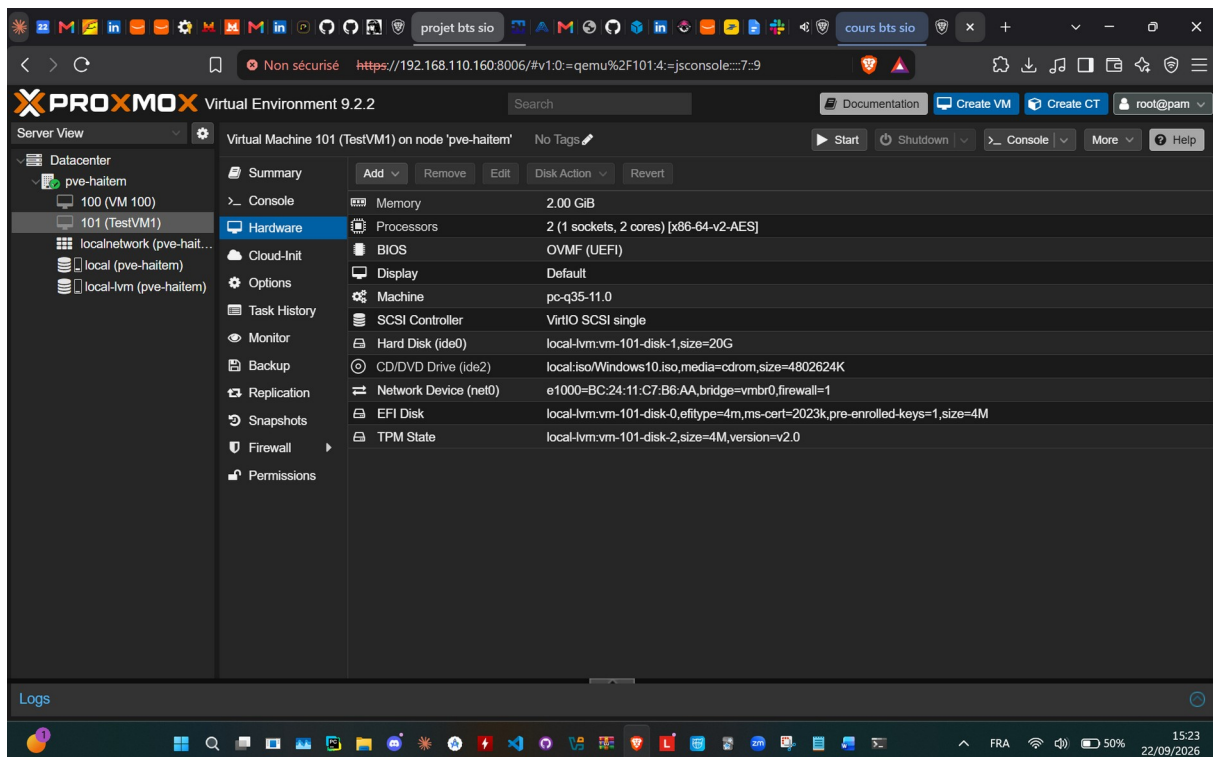
5.1 Objectif

Ajuster les ressources d'une VM en fonction de la charge, et maîtriser les snapshots.

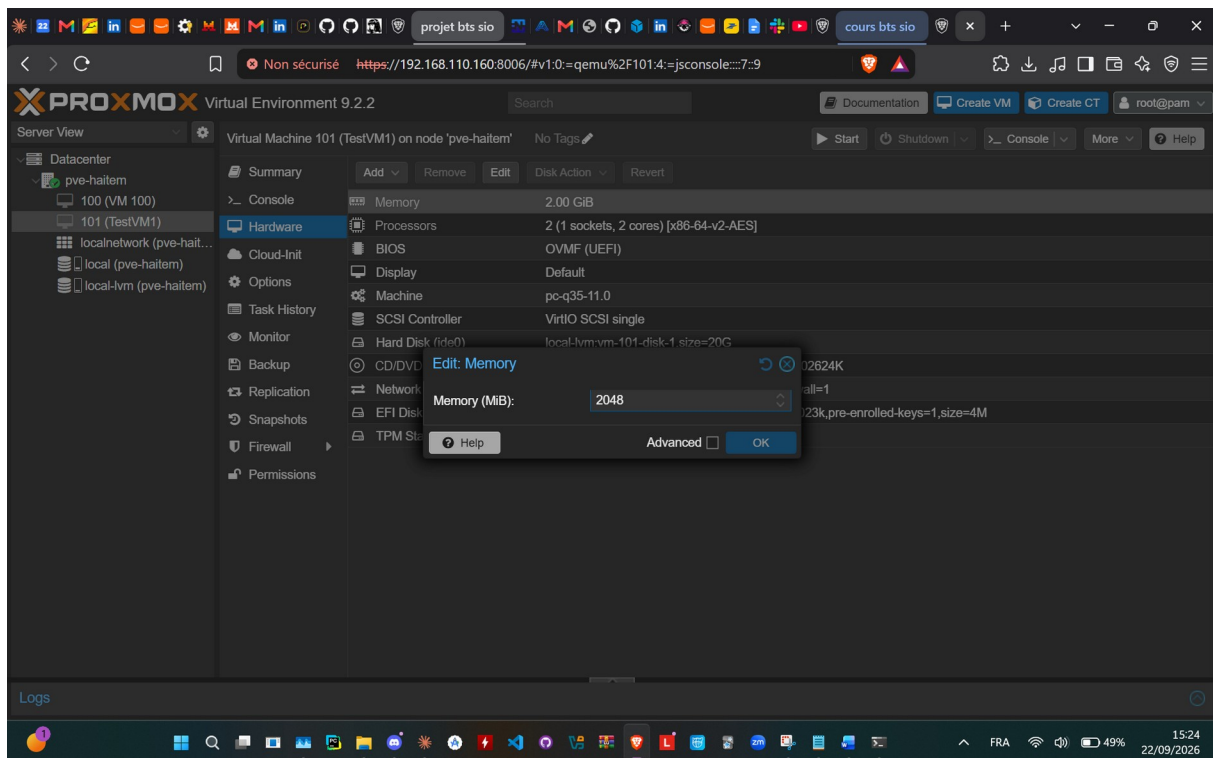
5.2 Augmentation de la RAM : +2 Go

La VM a d'abord été **arrêtée** : une modification de mémoire ne prend effet qu'au démarrage suivant, et on ne modifie pas le matériel d'une machine en fonctionnement.

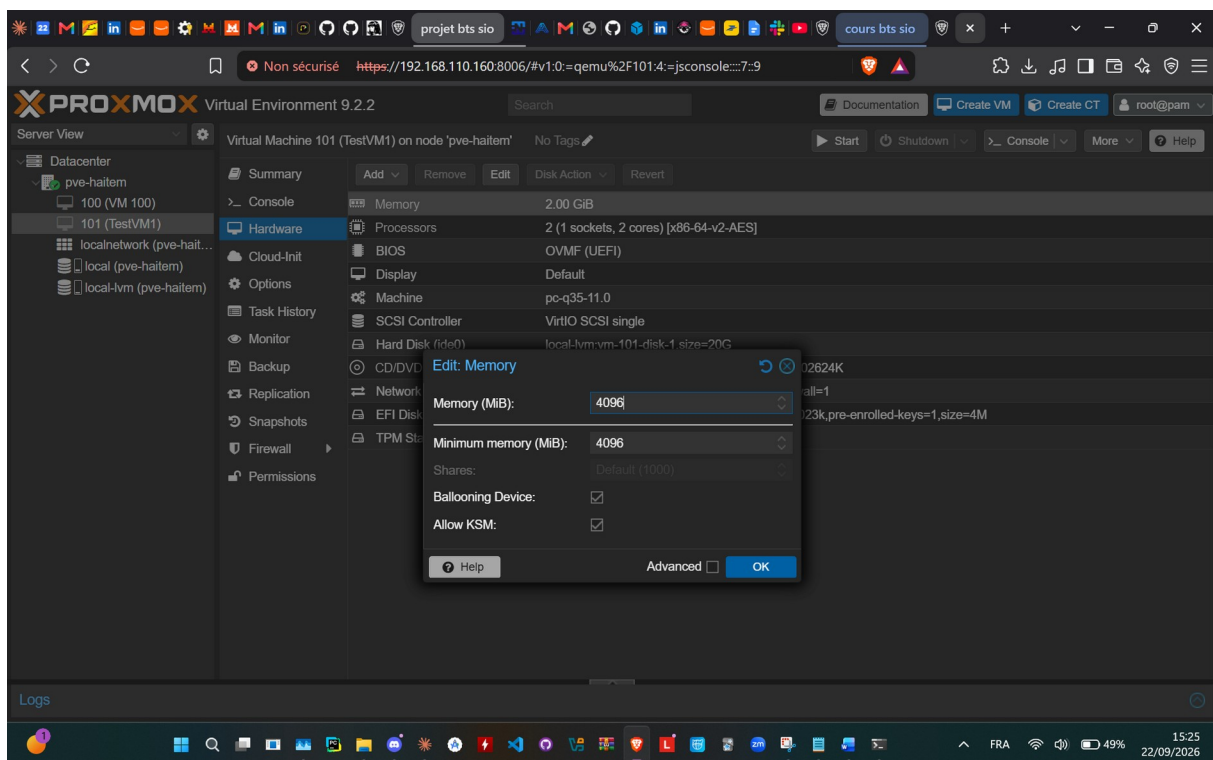
Cette modification a été réalisée **depuis l'interface web** (**Hardware** → double-clic sur **Memory** → **4096** → **OK**), afin de comparer ensuite avec la méthode en ligne de commande.



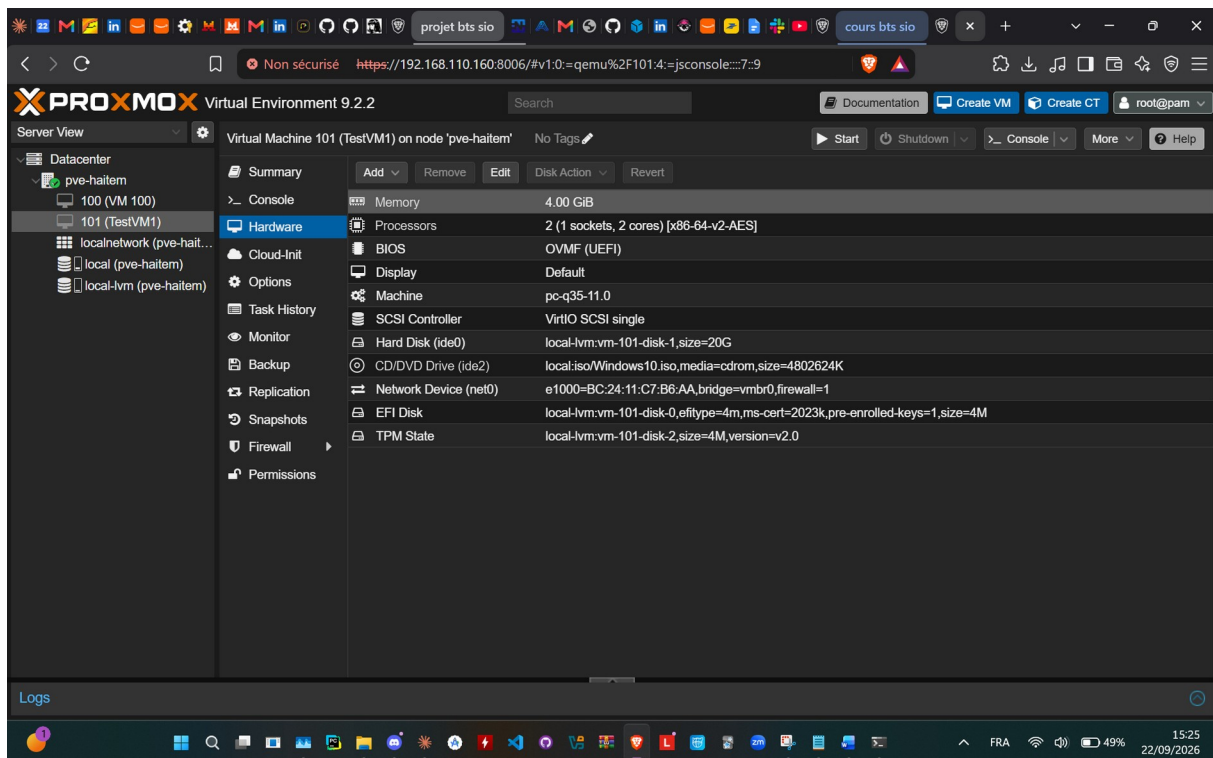
Capture 10a — **Memory 2.00 GiB, Processors 2 (1 sockets, 2 cores)**.



Capture 10b — *Edit: Memory*, valeur de départ 2048 MiB.



Capture 10c — valeur portée à 4096 MiB ; la fenêtre révèle les options de ballooning.



Capture 10d — Memory 4.00 GiB.

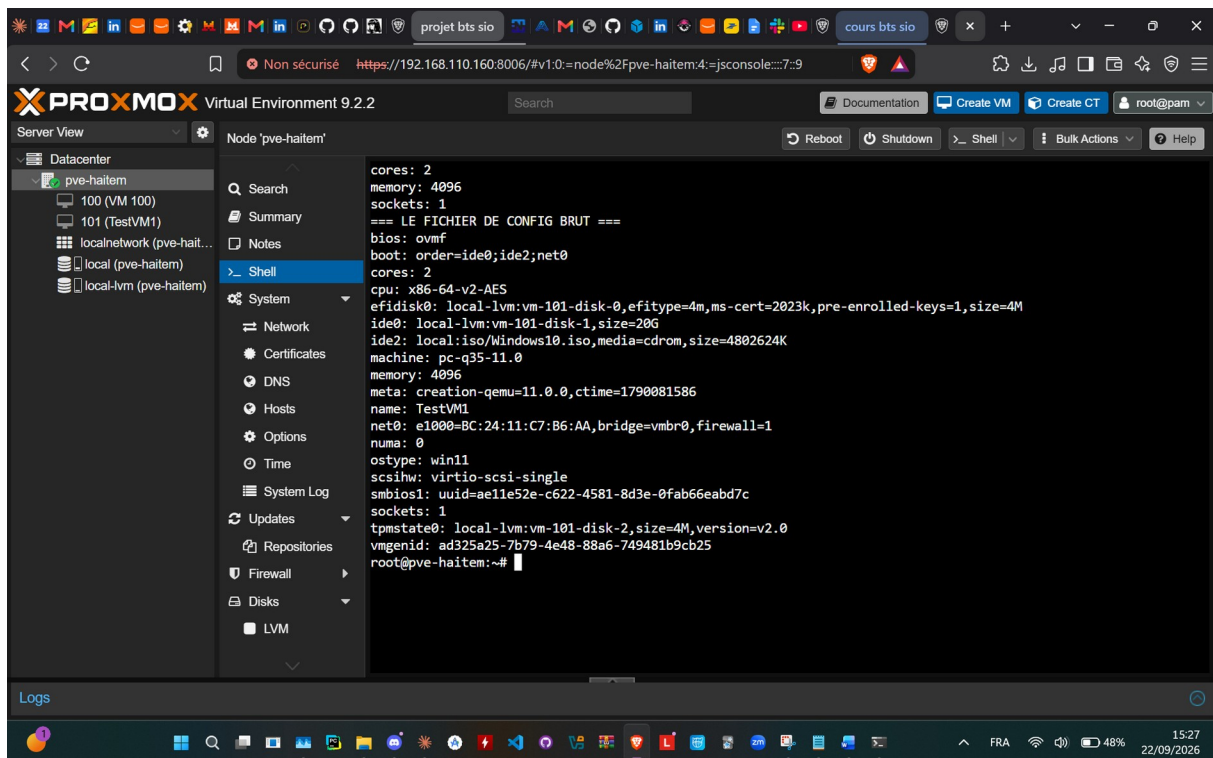
La saisie de la nouvelle valeur a fait apparaître trois options complémentaires :

- **Minimum memory : 4096** — aligné sur la valeur maximale, donc la VM conservera toujours ses 4 Gio. Une valeur inférieure autoriserait l'hyperviseur à lui en reprendre une partie sous pression.
- **Ballooning Device** ☒ — périphérique permettant cette récupération dynamique.
- **Allow KSM** ☒ — *Kernel Same-page Merging* : l'hôte fusionne les pages mémoire identiques entre plusieurs VM, très efficace lorsque plusieurs machines exécutent le même système.

5.3 Vérification : interface web et CLI écrivent au même endroit

```
qm config 101 | grep -E '^(memory|cores|sockets)'
```

```
cat /etc/pve/qemu-server/101.conf
```



Capture 11 — **memory: 4096** visible dans **qm config** et dans le fichier de configuration brut.

C'est le point central du TP. La configuration d'une VM n'est qu'un **fichier texte de 19 lignes** sur l'hyperviseur : **/etc/pve/qemu-server/101.conf**. L'interface web et la commande **qm** sont deux clients de la **même API REST**, qui écrivent dans ce même fichier :

Interface web ↗

↗ API REST Proxmox → /etc/pve/qemu-server/101.conf

Commande qm ↘

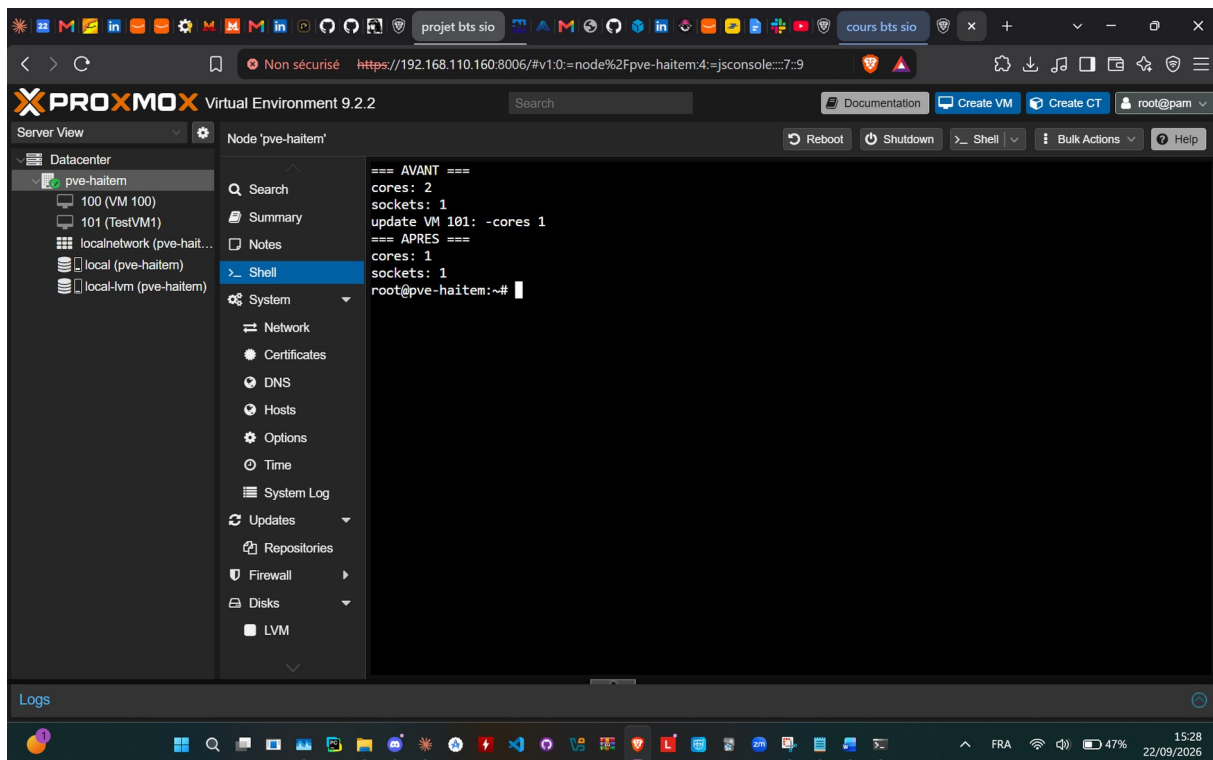
Modifier la RAM par un clic ou par **qm set 101 --memory 4096** produit donc un résultat rigoureusement identique. L'intérêt du CLI n'est pas le résultat, mais la **répétabilité** : traiter cinquante VM dans une boucle de script, ce qu'aucune interface graphique ne permet.

Cette parenté explique d'ailleurs un message d'erreur rencontré pendant le TP : une faute de frappe sur **qm status** a renvoyé **400 too many arguments** — un **code d'erreur HTTP** dans un terminal, parce que **qm** ne fait qu'appeler l'API.

5.4 Réduction du CPU : -1 cœur

Réalisée cette fois **en ligne de commande** :

```
qm set 101 --cores 1
```



Capture 12 — `cores: 2` avant, `cores: 1` après.

L'énoncé propose `cpu.total`, paramètre qui n'existe pas. Proxmox raisonne comme un serveur physique :

Paramètre	Signification
<code>--sockets</code>	nombre de processeurs physiques simulés
<code>--cores</code>	nombre de cœurs par socket
vCPU total	<code>sockets × cores</code>

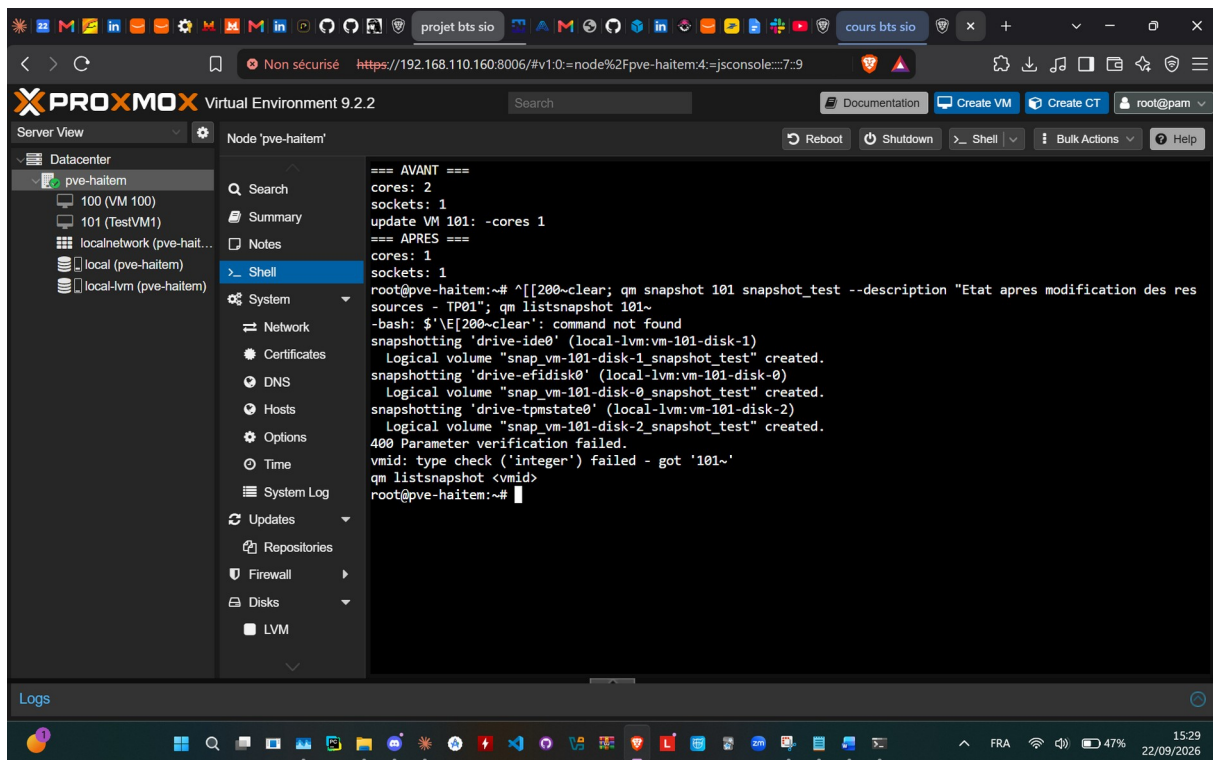
De $1 \times 2 = 2$ vCPU, la VM passe à $1 \times 1 = 1$ vCPU, soit bien -1 CPU.

La ligne `update VM 101: -cores 1` est la trace de la tâche API, également consultable dans l'onglet **Task History** de la VM.

Bilan des ressources : 2048 Mio / 2 vCPU → 4096 Mio / 1 vCPU.

5.5 Création d'un snapshot

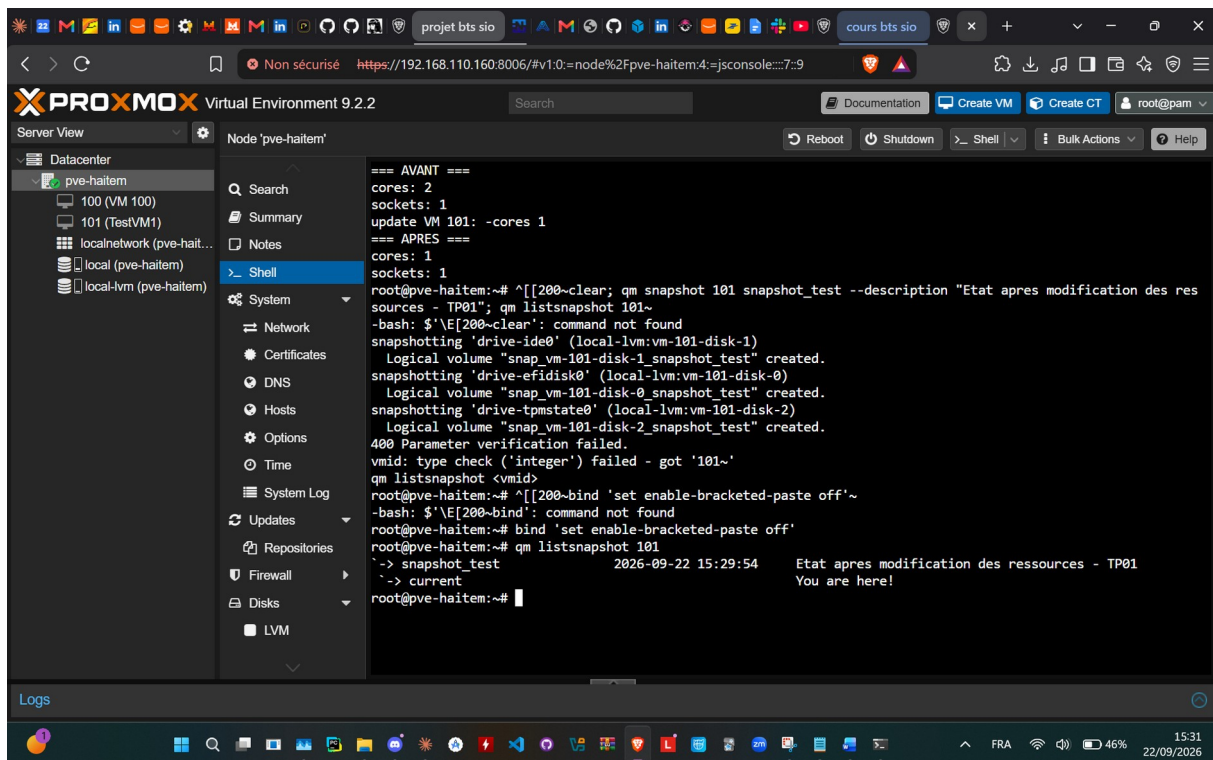
```
qm snapshot 101 snapshot_test --description "Etat apres modification des
ressources - TP01"
```



Capture 13a — création des volumes LVM du snapshot.

```
snapshotting 'drive-ide0' (local-lvm:vm-101-disk-1)
Logical volume "snap_vm-101-disk-1_snapshot_test" created.
snapshotting 'drive-efidisk0' (local-lvm:vm-101-disk-0)
Logical volume "snap_vm-101-disk-0_snapshot_test" created.
snapshotting 'drive-tpmstate0' (local-lvm:vm-101-disk-2)
Logical volume "snap_vm-101-disk-2_snapshot_test" created.
```

Analyse : Proxmox a créé un volume logique par disque virtuel — disque système, disque EFI et état du TPM. Un snapshot sur `local-lvm` n'est donc pas un fichier d'archive mais un **snapshot LVM-thin** géré par le noyau, en copie-sur-écriture. D'où son instantanéité : rien n'est copié à la création, seules les écritures ultérieures sont enregistrées à part.



Capture 13b — `qm listsnapshot 101`.

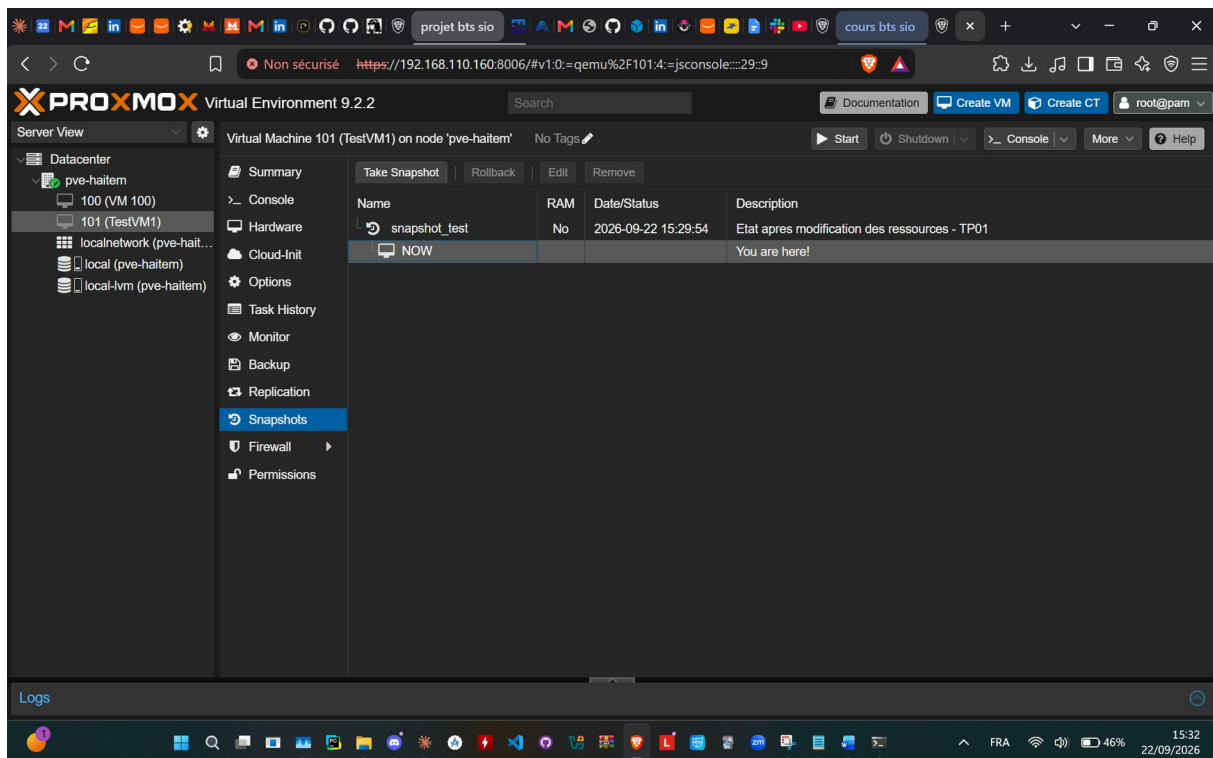
```

`-> snapshot_test      2026-09-22 15:29:54      Etat apres modification des
ressources - TP01

`-> current            You are here!

```

Les snapshots forment un **arbre** : **current** indique la position courante, au-dessus de l'état figé. Il est possible d'en créer plusieurs, de revenir à l'un d'eux et de repartir dans une autre direction.

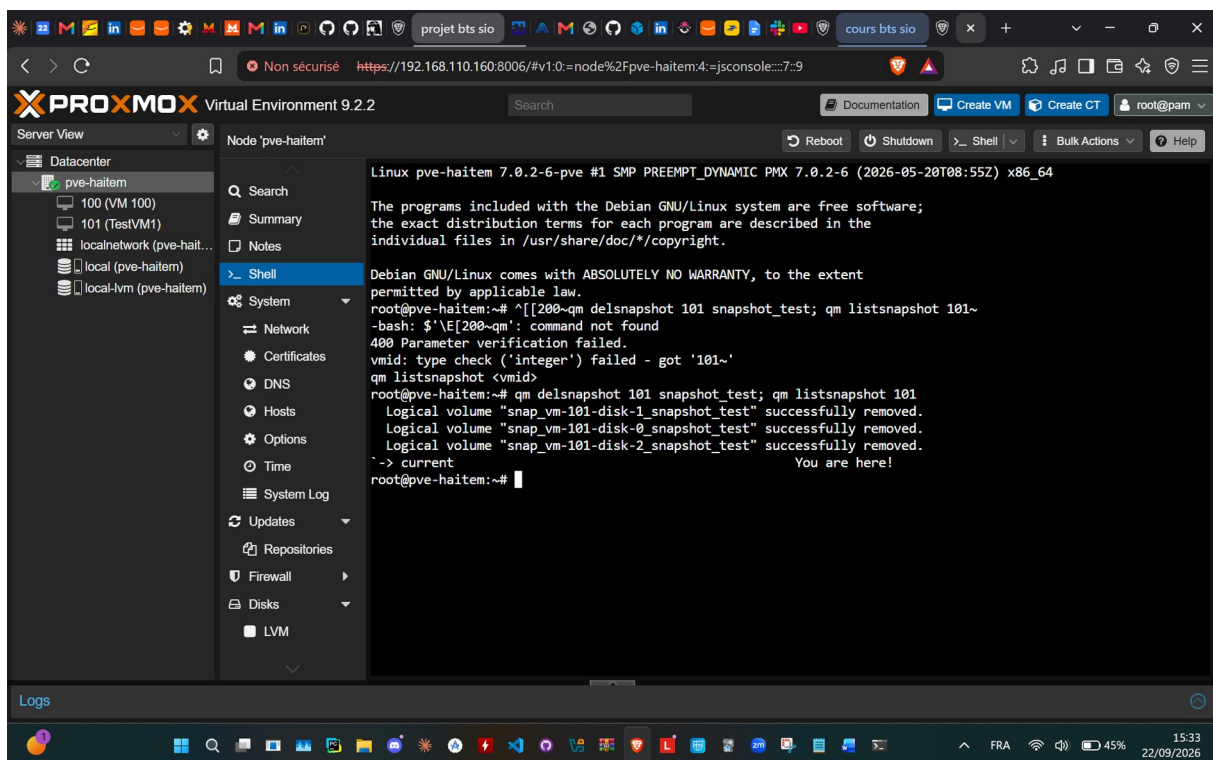


Capture 14 — le même snapshot vu depuis l'interface web.

La colonne **RAM : No** confirme que seul le disque a été capturé — la VM était éteinte. Sur une VM en fonctionnement, l'option *Include RAM* (`--vmstate 1`) capturerait aussi la mémoire, et la restauration rendrait la machine **en cours d'exécution**, processus compris.

5.6 Suppression du snapshot

```
qm delsnapshot 101 snapshot_test
```



Capture 15 — les trois volumes LVM sont détruits, seul **current** subsiste.

Cycle de vie complet d'un snapshot :

Action	Commande	Équivalent interface
Créer	<code>qm snapshot 101 <nom></code>	Snapshots → Take Snapshot
Lister	<code>qm listsnapshot 101</code>	onglet Snapshots
Restaurer	<code>qm rollback 101 <nom></code>	Rollback
Supprimer	<code>qm delsnapshot 101 <nom></code>	Remove

5.7 Critères de validation

- ☑ La RAM est augmentée de 2 Go (2048 → 4096 MiB)
- ☑ Le CPU est diminué de 1 (2 → 1 cœur)
- ☑ Un snapshot nommé **snapshot_test** est créé
- ☑ Le snapshot **snapshot_test** est supprimé

6. Incidents rencontrés et résolutions

#	Incident	Cause	Résolution
1	<code>ssh root@192.168.110.160</code> inaccessible	Service SSH injoignable depuis le poste	Utilisation du Shell intégré à l'interface web — terminal root équivalent
2	<code>can't lock file ... got timeout</code>	Verrou détenu par une tâche lancée depuis l'interface web	Vérification de l'absence de verrou persistant, puis reprise du cycle stop/start en CLI
3	Shutdown sans effet sur la VM	Aucun OS installé pour traiter le signal ACPI	Arrêt forcé par Stop — analysé comme un comportement attendu, pas comme une panne
4	<code>400 too many arguments</code>	Faute de frappe : <code>qm status 101 101--verbose</code>	Correction de la syntaxe ; l'occasion de constater que <code>qm</code> renvoie des codes HTTP
5	<code>--verbose \ head -20</code> inexploitable	Sortie triée alphabétiquement, les 20 premières lignes sont des sous-sections	Filtrage par <code>grep ^[a-z]</code> sur les champs de premier niveau
6	<code>\$('#E[200~clear]: command not found</code>	<i>Bracketed paste</i> : xterm.js encadre le collage de marqueurs que bash n'a pas filtrés	<code>bind 'set enable-bracketed-paste off'</code> , rendu permanent via <code>~/.inputrc</code>

7. Annexe A — Écarts entre l'énoncé et Proxmox VE 9

L'énoncé comporte des commandes et des éléments d'interface qui n'existent pas dans le produit. Les objectifs pédagogiques ont été atteints en utilisant les outils réellement fournis.

Énoncé	Réalité Proxmox VE 9
Onglet « Machines »	N'existe pas — bouton Create VM en haut à droite
« Sélectionner un modèle (Debian 11) »	La création exige une image ISO
« La VM démarre et affiche un bureau »	Impossible sans OS installé — on valide l'amorçage sur l'ISO
<code>pveccli / pvevm ...</code>	N'existe pas — l'outil est qm
<code>pvevm list --detail</code>	<code>qm config <vmid></code> et <code>qm status <vmid> --verbose</code>
<code>qm set TestVM1 ram.total 4096</code>	<code>qm set 101 --memory 4096</code>
<code>qm set TestVM1 cpu.total 2</code>	<code>qm set 101 --cores 1</code>
<code>qm snapshot create VM nom</code>	<code>qm snapshot 101 nom</code>
<code>qm snapshot delete VM nom</code>	<code>qm delsnapshot 101 nom</code>
<code>qm patch</code>	N'existe pas — voir <code>qm migrate</code> , <code>qm clone</code> , <code>qm template</code> , <code>qm resize</code>

8. Annexe B — Journal des commandes exécutées

Toutes les commandes lancées pendant le TP, dans l'ordre chronologique. Les actions réalisées dans l'interface graphique sont signalées en italique pour conserver la continuité du déroulé.

8.1 Préparation

#	Commande	Rôle
1	<code>pveversion</code>	Relever la version de Proxmox
2	<code>hostname</code>	Relever le nom du nœud
3	<code>qm list</code>	Inventorier les VM et repérer un VMID libre

```
pveversion; hostname; qm list
```

8.2 Exercice 1 — Interface web

Aucune commande : création via **Create VM**, démarrage par **Start**, arrêt par **Stop**, suppression de la VM **102** par **More** → **Remove**.

8.3 Exercice 2 — Ligne de commande

#	Commande	Rôle	Résultat
4	<code>qm list</code>	Lister les VM	VM 100 et 101 , toutes deux stopped
5	<code>qm start 101; qm status 101</code>	Démarrer puis vérifier	✗ can't lock file ... got timeout
6	<code>qm config 101 \\\ grep -i '^lock'</code>	Chercher un verrou persistant	aucune sortie → pas de verrou bloqué
7	<code>qm stop 101</code>	Arrêter en CLI	succès (aucune sortie)
8	<code>sleep 5</code>	Laisser le verrou se libérer	—
9	<code>qm start 101</code>	Démarrer en CLI	swtpm_setup: Not overwriting existing state file.

#	Commande	Rôle	Résultat
10	<code>qm status 101</code>	Vérifier	<code>status: running</code>
11	<code>qm config 101</code>	Configuration matérielle complète	19 lignes
12	<code>qm status 101 --verbose \ head -20</code>	État d'exécution	✗ inexploitable (sous-sections uniquement)
13	<code>qm status 101 --verbose \ grep ^[a-z]</code>	État d'exécution filtré	✓ champs de premier niveau

```
qm list
qm start 101; qm status 101
qm config 101 | grep -i '^lock'; qm stop 101; sleep 5; qm start 101; qm status 101
qm config 101
qm status 101 --verbose | grep ^[a-z]
```

8.4 Exercice 3 — Ressources et snapshots

#	Commande	Rôle	Résultat
14	<i>Interface : <code>Hardware</code> → <code>Memory</code> → <code>4096</code></i>	Augmenter la RAM de 2 Go	<code>Memory 4.00 GiB</code>
15	<code>qm config 101 \ grep -E '^(memory\ cores\ sockets)'</code>	Vérifier la modification	<code>memory: 4096</code>
16	<code>cat /etc/pve/qemu-server/101.conf</code>	Lire le fichier de configuration brut	<code>memory: 4096</code> confirmé
17	<code>qm set 101 --cores 1</code>	Réduire le CPU d'un cœur	<code>update VM 101: -cores 1</code>

#	Commande	Rôle	Résultat
18	<code>qm snapshot 101 snapshot_test --description "..."</code>	Créer un snapshot	3 volumes LVM créés
19	<code>qm listsnapshot 101</code>	Lister les snapshots	<code>snapshot_test + current</code>
20	<code>qm delsnapshot 101 snapshot_test</code>	Supprimer le snapshot	3 volumes LVM détruits
21	<code>qm list</code>	Vérification finale	<code>100</code> et <code>101, 101</code> à 4096 Mo

Vérification de la modification faite dans l'interface

```
qm config 101 | grep -E '^(memory|cores|sockets)'
```

```
cat /etc/pve/qemu-server/101.conf
```

Réduction du CPU

```
qm config 101 | grep -E '^(cores|sockets)'
```

```
qm set 101 --cores 1
```

```
qm config 101 | grep -E '^(cores|sockets)'
```

Snapshots

```
qm snapshot 101 snapshot_test --description "Etat apres modification des ressources - TP01"
```

```
qm listsnapshot 101
```

```
qm delsnapshot 101 snapshot_test
```

```
qm listsnapshot 101
```

Contrôle final

```
qm list
```

8.5 Commandes d'environnement (hors exercices)

Commandes liées à la configuration du poste de travail, sans rapport avec les critères de validation mais nécessaires au bon déroulement du TP :

#	Commande	Rôle
22	<code>bind 'set enable-bracketed-paste off'</code>	Corriger le collage dans le Shell — valable pour la session en cours
23	<code>echo 'set enable-bracketed-paste off' >> ~/.inputrc</code>	Rendre le correctif permanent

8.6 Séquence complète, prête à rejouer

L'intégralité du TP en ligne de commande, hors actions graphiques :

```
# --- Préparation ---
pveversion; hostname; qm list

# --- Exercice 2 : pilotage en ligne de commande ---
qm list                    # lister les VM
qm stop 101                # arrêter
qm start 101               # démarrer
qm status 101              # statut
qm config 101               # configuration (statique)
qm status 101 --verbose | grep ^[a-z]  # état d'exécution (dynamique)

# --- Exercice 3 : ressources ---
qm set 101 --memory 4096    # RAM : 2048 -> 4096 MiB (+2 Go)
qm set 101 --cores 1        # CPU : 2 -> 1 coeur (-1 vCPU)
qm config 101 | grep -E '^(memory|cores|sockets) '

# --- Exercice 3 : snapshots ---
qm snapshot 101 snapshot_test --description "Etat apres modification des
ressources - TP01"
```

```
qm listsnapshot 101
qm delsnapshot 101 snapshot_test
qm listsnapshot 101

# --- Contrôle final ---
qm list
```

La commande `qm set 101 --memory 4096` figure ici pour rendre la séquence rejouable de bout en bout. Pendant le TP, cette modification a été réalisée depuis l'interface graphique, afin de démontrer que les deux méthodes écrivent dans le même fichier de configuration (§ 5.3).

9. Conclusion

Les douze critères de validation des trois exercices sont atteints. La VM `101 / TestVM1` est conservée en l'état pour servir de source de sauvegarde au TP03.

Ce qui ressort du TP :

1. **L'interface web et la ligne de commande ne sont pas deux produits, mais deux clients de la même API REST.** Vérifié concrètement : une modification faite par clic apparaît immédiatement dans le fichier de configuration lu par `qm`.
2. **Une machine virtuelle est une description textuelle.** Dix-neuf lignes suffisent à définir le matériel complet d'un serveur — ce qui explique la facilité à cloner, sauvegarder ou migrer.
3. **Le choix de l'outil dépend du volume et de la répétabilité,** pas du résultat : l'interface pour une intervention ponctuelle, le CLI dès qu'il faut scripter ou traiter plusieurs machines.
4. **Un énoncé technique se vérifie.** La moitié des commandes fournies étaient inexactes ; les identifier et retrouver les commandes réelles dans la documentation constitue une part significative du travail d'administration système.

Compétences mobilisées (BTS SIO — bloc *Support et mise à disposition de services informatiques*)

Compétence	Mise en œuvre
Gérer le patrimoine informatique	Inventaire des VM, relevé de la configuration matérielle virtuelle
Mettre à disposition un service informatique	Provisionnement d'une VM de test, allocation des ressources

Compétence	Mise en œuvre
Traiter des demandes concernant les services	Ajustement de la RAM et du CPU en fonction du besoin
Développer la présence en ligne / Travailler en mode projet	<i>(non concerné par ce TP)</i>
Répondre à un incident	Diagnostic du verrou, de l'arrêt ACPI et du bracketed paste

Compte-rendu réalisé le 22/09/2026 — Proxmox VE 9.2.2, nœud *pve-haitem*.